UNIT – IV: AGILE TESTING

1. Identify the steps involved in performing Risk-Based Testing.

Risk-Based Testing (RBT) is a testing strategy in which testing effort is given according to the **risk level** of different parts of the system. High-risk and high-impact features receive more testing attention.

Steps involved:

1. **Identify the features**
    - List all features or components planned for the sprint.
2. **Assess the risk**
    - For each feature, estimate:
        - **Probability of failure**
        - **Impact/Severity** if failure occurs.
3. **Calculate overall risk**
    - Risk is commonly considered as:

    Risk = Probability × Impact

4. **Rank the features**
    - Arrange features from **highest risk to lowest risk** using a risk matrix.
5. **Allocate testing effort**
    - High-risk features receive:
        - More test scenarios
        - More exploratory testing
        - Higher automation priority.
6. **Reassess risk**
    - Risk assessment is updated during the sprint when new information or defects are discovered.

Factors affecting risk:

- Business-critical functionality
- Frequently changing areas
- Technical complexity
- Historical defect density
- Regulatory or compliance requirements.

Conclusion: Risk-Based Testing helps Agile teams use limited testing time effectively by focusing on the areas with the highest probability and impact of failure.

---

2. Describe the role of scenarios in Agile Testing.

In Agile, a **user story** describes a feature from the user's point of view. Acceptance criteria define the conditions that must be satisfied for the story to be complete. **Scenarios convert these criteria into concrete and testable situations.**

Role of scenarios:

1. **Remove ambiguity**
    - They convert general requirements into precise and checkable conditions.
2. **Define expected behaviour**
    - They clearly explain what should happen for a particular situation.
3. **Support acceptance testing**
    - Acceptance tests are created from the acceptance criteria of a user story.
4. **Use Given-When-Then format**
    - **Given** – starting condition
    - **When** – action performed
    - **Then** – expected result.
5. **Help find missing requirements**
    - Writing scenarios can reveal incomplete or conflicting requirements early.
6. **Support automation**
    - Scenarios can often be directly automated using BDD tools such as Cucumber.
7. **Create shared understanding**
    - Developers, testers, customers and other stakeholders get a common understanding of what "done" means.

Example:

User Story:
As a customer, I want to apply a discount coupon so that I can pay less.

Scenario:

- Given the customer has items worth ₹2000
- When the customer applies a valid 10% coupon
- Then the order total should become ₹1800.

Conclusion: Scenarios make Agile requirements clear, testable and understandable to the entire team.

---

3. What is Exploratory Testing? Mention its advantages in Agile Projects.

Exploratory Testing is an approach where **test design, learning and test execution happen at the same time**. The tester does not depend completely on pre-written test cases.

How it works:

The tester:

1. Learns about the system.
2. Designs tests based on what is learned.
3. Executes the tests.
4. Uses the results to decide what to test next.

Advantages in Agile:

1. **Suitable for changing requirements**
   - Agile requirements change frequently, so detailed scripted tests may become outdated.
2. **Finds unexpected defects**
   - Testers can discover problems that scripted tests may not cover.
3. **Useful for new features**
   - It is helpful when a feature is new or not fully understood.
4. **Finds usability issues**
   - Testers can identify problems related to user experience.
5. **Finds edge cases**
   - Testers can try unusual inputs and situations.
6. **Checks feature interactions**
   - It can discover problems caused when different features work together.

Session-Based Test Management:

To make exploratory testing disciplined, teams may use **time-boxed sessions**, usually 60–90 minutes, with a testing **charter** describing what should be explored.

Exploratory vs Scripted Testing:

Scripted Testing	Exploratory Testing
Tests are planned in advance	Tests are designed during execution
Highly repeatable	Less repeatable
Good for stable requirements	Good for new/evolving requirements
Detailed test documents	Lightweight charters and notes

Conclusion: Exploratory testing complements automated and scripted testing and is very useful for finding unexpected problems in Agile projects.

---

4. Explain any four tools/frameworks used for TDD and state their purpose.

xUnit is a family of unit-testing frameworks used to create and execute automated unit tests. Different programming languages have different xUnit tools.

1. JUnit

- Used for **Java**.
- One of the most widely used Java unit-testing frameworks.
- Supports annotations such as `@Test` and `@BeforeEach`.

2. NUnit

- Used for **.NET applications**, especially C# and VB.NET.
- Closely modelled on JUnit.
- Integrates with Visual Studio.

3. PyUnit / unittest

- Used for **Python**.
- `unittest` is included with Python.
- Used to create and run unit tests.

4. Jest

- Used for **JavaScript/TypeScript**.
- Provides a test runner, assertion library and mocking support.

Other tools:

- **RSpec** – Ruby testing using behaviour-driven style.
- **Mockito** – Java mocking library used with JUnit to simulate dependencies.

Common xUnit concepts:

- **Test Case** – tests one specific behaviour.
- **Test Fixture** – fixed state/data used for testing.
- **setUp()/tearDown()** – prepare and clean up before/after tests.
- **Assertions** – compare actual and expected results.
- **Test Suite** – collection of test cases.
- **Test Runner** – executes tests and reports results.

Conclusion: These frameworks make TDD practical by allowing developers to write and automatically execute unit tests.

---

5. Differentiate between Agile Testing and Traditional Software Testing.

Traditional Testing	Agile Testing
Testing is generally a separate phase	Testing is a continuous activity
Usually starts after coding is largely complete	Starts from the beginning

Traditional Testing	Agile Testing
Testers work mainly as a separate QA group	Developers, testers and business analysts share responsibility
Feedback may come late	Feedback is received quickly
Large testing phase may occur near the end	Testing happens in every sprint
Regression testing may be less frequent	Regression testing is performed continuously
More manual testing may be involved	Strong emphasis on automation
Requirements are relatively stable	Requirements can evolve frequently

Agile Testing characteristics:

1. Testing is performed throughout every iteration.
2. Acceptance criteria and test scenarios are discussed before coding.
3. Defects are found and fixed quickly.
4. Regression testing is performed after changes.
5. Automated unit, API and UI tests are encouraged.
6. Testers participate in story discussions and exploratory testing.

Conclusion: Traditional testing is mainly a separate later activity, whereas Agile testing is continuous, collaborative and strongly supported by automation.

---

6. Which types of tests are most suitable for automation in Agile Projects?

Test automation means using software to execute tests, compare actual and expected results and report the outcome with minimum manual intervention. Automation is important in Agile because tests must be repeated frequently.

Tests suitable for automation:

1. Unit Tests

- Test individual functions or classes.
- Very fast and inexpensive.
- Should form the largest part of automated tests.
- TDD produces many automated unit tests.

2. Integration/API Tests

- Check interaction between components, services or APIs.
- Useful for verifying communication between different parts of the system.

3. Regression Tests

- Previously passed tests can

- This helps ensure existing functionality is not broken.

4. Data-Driven Tests

- Useful when the same functionality must be tested with many input combinations.

5. Business-Critical Flows

- Important flows such as payment or checkout can be automated because they are frequently tested.

Test Automation Pyramid:

Top: Few UI/End-to-End tests
Middle: Moderate Integration/API tests
Bottom: Many Unit tests

The PDF recommends many fast unit tests, fewer integration tests and only a small number of UI/end-to-end tests.

Better kept manual:

- Brand-new changing features
- Usability and look-and-feel testing
- Highly exploratory investigations.

Conclusion: Automation is best for stable, repetitive and frequently executed tests, while human judgement is still important for exploratory and usability testing.

---

7. How are User Stories converted into Acceptance Tests?

A **user story** is a short description of a feature from the end user's perspective. Since a story alone is incomplete, the team and customer agree on **acceptance criteria** that define when the story is complete.

Steps to convert a User Story into Acceptance Tests:

1. **Write the User Story**
    - Example:
      "As a customer, I want to apply a discount coupon so that I can pay a reduced price."
2. **Identify Acceptance Criteria**
    - Define specific conditions that the feature must satisfy.
3. **Create Test Scenarios**
    - Convert each acceptance criterion into one or more test situations.

4. **Use Given-When-Then**
    - **Given** – initial condition
    - **When** – user action
    - **Then** – expected result.
5. **Automate the scenarios when possible**
    - BDD tools such as Cucumber can convert these scenarios into executable tests.

Example:

Acceptance Criterion: A valid 10% coupon should reduce a ₹2000 order to ₹1800.

Acceptance Test:

- **Given:** Customer has items worth ₹2000.
- **When:** Customer applies the valid coupon `SAVE10`.
- **Then:** Order total should be ₹1800.

Another test can check an expired coupon and verify that it is rejected.

Benefits:

- Removes ambiguity.
- Defines clear expected behaviour.
- Helps identify missing requirements early.
- Can be automated.
- Gives the team a common definition of "done".

Conclusion: User stories are converted into acceptance tests by defining acceptance criteria and expressing them as clear, testable scenarios.

---

8. Describe the Red-Green-Refactor cycle of TDD.

Test-Driven Development (TDD) is a practice where the developer writes an automated test **before writing the production code**. The main TDD cycle is **Red → Green → Refactor**.

1. RED – Write a failing test

- Write the smallest possible test for a new behaviour.
- The required code does not exist yet.
- Run the test and it fails.
- This is called the **Red** state.

2. GREEN – Make the test pass

- Write the simplest code necessary to make the test pass.
- Do not focus on perfect design at this stage.

- Run the tests again.
- When they pass, the system is in the **Green** state.

3. REFACTOR – Improve the code

- Clean and improve the code without changing its behaviour.
- Remove duplication.
- Improve names and simplify logic.
- Run the tests continuously to make sure nothing breaks.

Diagram:

RED
↓
Write failing test
↓
GREEN
↓
Write minimum code to pass
↓
REFACTOR
↓
Improve code
↓
Repeat the cycle

Advantages:

- Encourages better design.
- Produces automated regression tests.
- Finds defects early.
- Makes refactoring safer.
- Produces loosely coupled and testable code.

Conclusion: TDD continuously builds both the software and its automated test suite through the repeated **Red → Green → Refactor** cycle.